INSTITUTO FEDERAL DO CEARÁ (IFCE) • DISCIPLINA DE ANÁLISE E PROJETO DE SISTEMAS (APS)

Amanda Cavalcante Vieira
Isaac da Cunha Cavalcanti Duarte
João Pedro Moreira Campos de Lima
Rodrigo Sousa Sales

GameKube

Plataforma Web para Execução de Jogos em Contêineres Orquestrados por Kubernetes

ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE (SRS)

Documento elaborado conforme as diretrizes simplificadas da norma IEEE 830.
Referência: Semestre 2026.2 • Setembro de 2026

1. Sumário

1. Sumário **2**

2. Introdução **3**

2.1 Finalidade do Documento 3

2.2 Escopo do Projeto 3

2.3 Definições, Acrônimos e Abreviações 3

2.4 Visão Geral da Arquitetura e Tecnologias 4

3. Modelo de Casos de Uso (UML) **5**

3.1 Atores do Sistema 5

3.2 Diagrama Geral de Casos de Uso 5

3.3 Especificação dos Casos de Uso Principais 6

4. Requisitos Funcionais **7**

4.1 Módulo de Autenticação e Gestão de Identidade 7

4.2 Módulo de Gestão de Contas e Perfis de Usuário 8

4.3 Módulo de Catálogo e Execução de Jogos Sob Demanda 9

4.4 Módulo Social (Amizades e Diretório) 10

4.5 Módulo de Presença em Tempo Real e Mensageria 11

4.6 Matriz de Resumo e Priorização (MoSCoW) 12

5. Requisitos Não-Funcionais **13**

5.1 Desempenho e Eficiência 13

5.2 Segurança e Privacidade 13

5.3 Confiabilidade, Disponibilidade e Tolerância a Falhas 14

5.4 Usabilidade e Acessibilidade 14

5.5 Portabilidade e Manutenibilidade da Infraestrutura 15

5.6 Matriz de Resumo dos Requisitos Não-Funcionais 15

2. Introdução

2.1 Finalidade do Documento

Este documento apresenta a **Especificação de Requisitos de Software (SRS - Software Requirements Specification)** para a plataforma **GameKube**. O propósito é definir de forma clara e verificável os serviços (requisitos funcionais) e as restrições operacionais e arquiteturais (requisitos não-funcionais) do sistema, servindo como guia oficial para o ciclo de desenvolvimento, testes e homologação na disciplina de Análise e Projeto de Sistemas (APS - 2026.2).

2.2 Escopo do Projeto

O **GameKube** é uma plataforma web para execução de jogos em contêineres orquestrados por Kubernetes. A aplicação elimina as barreiras clássicas do modelo tradicional de jogos (instalação manual, compatibilidade com sistemas operacionais e exigências pesadas de hardware do cliente), permitindo que jogadores autenticados iniciem instâncias de jogos diretamente no navegador com isolamento estrito de recursos.

O escopo abrange quatro pilares centrais:

- **Execução de Jogos Sob Demanda:** Criação dinâmica de Pods, Services, Ingress TLS e OAuth2-Proxy no Kubernetes com encerramento automático por ociosidade (idle timeout);
- **Camada Social e Comunidade:** Diretório de jogadores com busca paginada, sistema bilateral de amizades (solicitar, aceitar, recusar e remover) e chat em tempo real via STOMP/WebSocket;
- **Presença em Tempo Real:** Controle de status (Online, Ocupado, Ausente, Offline) e indicação do jogo em execução;
- **Identidade e Segurança:** Autenticação centralizada com Keycloak (OIDC/PKCE) e controle de acesso baseado em papéis (RBAC).

2.3 Definições, Acrônimos e Abreviações

Termo / Acrônimo	Definição
APS	Análise e Projeto de Sistemas.
SRS / ERS	Software Requirements Specification (Especificação de Requisitos de Software).
Kubernetes (K8s)	Orquestrador de contêineres para automação de implantação, escalabilidade e gerenciamento.
Pod	Menor unidade de computação no K8s, encapsulando os contêineres de cada sessão de jogo.
Keycloak / OIDC	Servidor de identidade e acesso utilizando OpenID Connect sobre OAuth 2.0 com fluxo PKCE.
STOMP	Simple Text Oriented Messaging Protocol; protocolo de mensagens sobre conexões WebSocket.
Idle Timeout	Mecanismo de coleta e destruição automática de sessões inativas (5 minutos sem heartbeat).
MoSCoW	Método de priorização: *Must have* (Obrigatório), *Should have* (Importante), *Could have* (Desejável).

2.4 Visão Geral da Arquitetura e Tecnologias

Camada	Tecnologias	Papel no Sistema
Frontend (Web)	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, Radix UI	Interface gráfica responsiva, catálogo, terminal de jogo (game stage) e chat social.
Backend (Aplicação)	Java 21/25, Spring Boot 4.1, Spring Security, Spring STOMP, Fabric8 K8s	Regras de negócio, orquestração de pods no K8s, controle de amizades e agendador.
Identidade (IAM)	Keycloak 25/26 (OpenID Connect / OAuth 2.0)	Autenticação centralizada, emissão de tokens JWT e gestão de credenciais.
Banco de Dados	PostgreSQL 16/18, Flyway Migration	Persistência relacional de usuários, solicitações de amizade e histórico de mensagens.
Cache e Presença	Redis 7/8	Armazenamento em memória para estados de presença e cache de catálogo de jogos.
Orquestração / Infra	Kubernetes, Helm, Traefik Ingress, Cert-Manager	Provisionamento dinâmico de instâncias, roteamento seguro e certificados TLS.

3. Modelo de Casos de Uso (UML)

3.1 Atores do Sistema

- **Visitante:** Usuário não autenticado que acessa a landing page e pode realizar login ou cadastro;
- **Jogador:** Usuário autenticado que navega pelo catálogo, joga partidas sob demanda, interage socialmente com amigos e configura sua conta;
- **Orquestrador Kubernetes:** Ator autônomo que instancia, monitora via probes de prontidão e destrói contêineres de jogos;
- **Administrador:** Usuário com privilégios para telemetria e gestão do catálogo de jogos.

3.2 Diagrama Geral de Casos de Uso

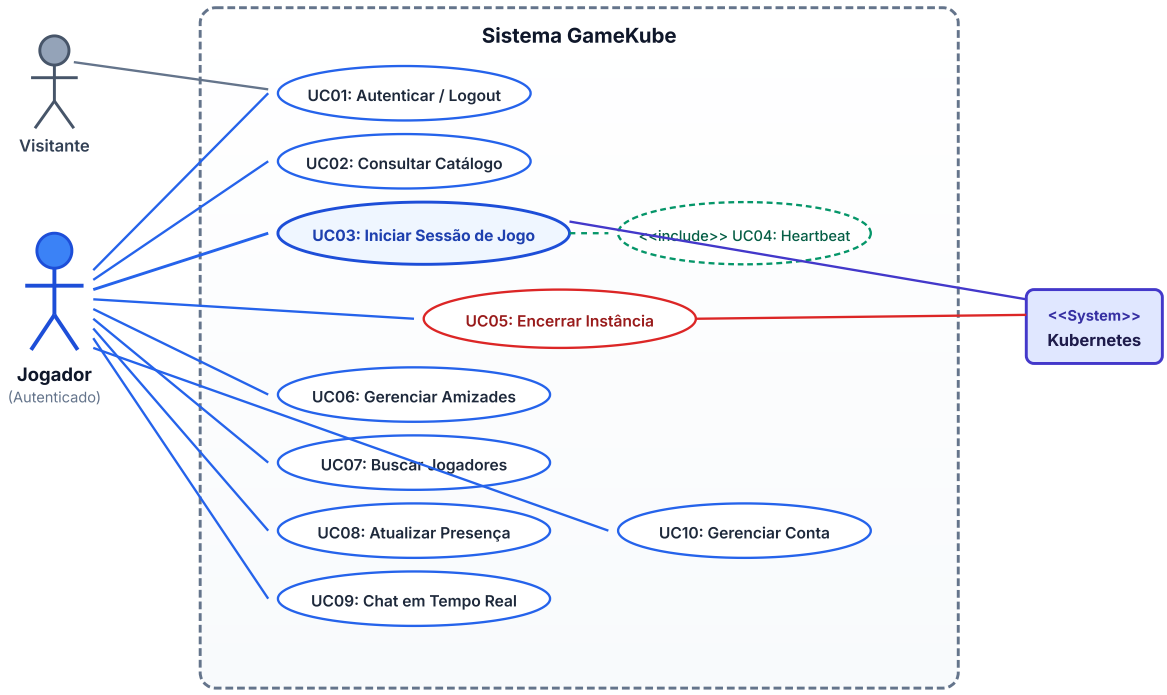


Figura 1: Diagrama Geral de Casos de Uso do Sistema GameKube (UML).

3.3 Especificação dos Casos de Uso Principais

Caso de Uso	Descrição e Fluxos
UC01: Autenticar / Logout	**Atores:** Visitante, Jogador, Keycloak. **Fluxo:** Redirecionamento com desafio PKCE, autenticação no Keycloak, emissão de tokens JWT e cookie cifrado. Ao deslogar, invalidação de sessão e presença `OFFLINE`.
UC03: Iniciar Sessão de Jogo	**Atores:** Jogador, Orquestrador Kubernetes. **Fluxo:** O jogador escolhe um título; o backend cria Deployment, Service, Ingress e Secret TLS no K8s; aguarda *readiness probes* e entrega URL segura para o *Game Stage*.
UC04: Manter Heartbeat / Limpeza	**Atores:** Jogador (Frontend), Agendador (Backend), Kubernetes. **Fluxo:** Envio de pulso a cada 15s. Ausência de pulso por 5 min (idle timeout) aciona a destruição automática do Pod no cluster.
UC06: Gerenciar Amizades	**Atores:** Jogadores. **Fluxo:** Envio de solicitação bilateral, notificações de pendência, aceite via PATCH ou recusa, e listagem em tempo real.
UC08: Atualizar Presença	**Atores:** Jogador, Redis, WebSocket. **Fluxo:** Jogador altera status (Online, Busy, Away, Offline). Atualização no Redis e disparo imediato via STOMP para a lista de amigos.

4. Requisitos Funcionais

4.1 Módulo de Autenticação e Gestão de Identidade

RF01 Autenticação Centralizada e SSO (OIDC/PKCE) — MUST HAVE

O sistema deve autenticar usuários centralizadamente com OpenID Connect (OIDC) e fluxo PKCE via Keycloak, emitindo tokens JWT e cookies de sessão criptografados.

RF02 Cadastro de Nova Conta de Usuário — MUST HAVE

O sistema deve permitir o autorregistro com nome de usuário único, e-mail válido, nome completo e senha protegida.

RF03 Controle de Acesso Baseado em Papéis (RBAC) — MUST HAVE

O sistema deve restringir operações e rotas através de anotações de autorização (`@PreAuthorizeUser` e `@PreAuthorizeAdmin`).

RF04 Encerramento Seguro de Sessão (Logout) — MUST HAVE

O sistema deve revogar tokens, limpar cookies de sessão e alterar a presença do usuário para `OFFLINE` no encerramento.

4.2 Módulo de Gestão de Contas e Perfis de Usuário

RF05 Visualização e Edição de Perfil Pessoal — MUST HAVE

O usuário autenticado pode consultar e atualizar seus próprios dados de perfil (nome, bio, preferências visuais).

RF06 Alteração de E-mail com Reautenticação — SHOULD HAVE

Permite a alteração do endereço de e-mail mediante confirmação da senha atual.

RF07 Exclusão Permanente de Conta (LGPD) — SHOULD HAVE

Permite a remoção definitiva da conta, limpando credenciais, presenças e histórico associado.

4.3 Módulo de Catálogo e Execução de Jogos Sob Demanda

RF08 Consulta ao Catálogo de Jogos — MUST HAVE

Exibição de jogos disponíveis com capa, sinopse, gênero, controles e requisitos técnicos cacheados no Redis.

RF09 Provisionamento Dinâmico de Instância K8s — MUST HAVE

Criação sob demanda de Deployment, Service, Ingress e autenticação OAuth2-Proxy isolada por partida.

RF10 Acompanhamento de Prontidão (Readiness Probes) — MUST HAVE

Monitoramento do Pod no K8s até estar apto a receber conexões (timeout configurado para 5 minutos).

RF11 **Ambiente de Execução Embutido (Game Stage)** MUST HAVE

Interface dedicada (`/play/[slug]`) com toolbar, modo tela cheia, indicadores de status e controle de sessão.

RF12 **Monitoramento de Heartbeat e Idle Cleanup** MUST HAVE

Emissão periódica de pulso de atividade (15s) e destruição automática de pods ociosos após 5 minutos.

RF13 **Encerramento Manual de Instância** MUST HAVE

Botão para encerrar a partida imediatamente e desalocar os recursos no Kubernetes.

4.4 Módulo Social (Amizades e Diretório)

RF14 **Diretório de Jogadores Paginado** — MUST HAVE

Listagem e busca de jogadores cadastrados com suporte a filtros e paginação.

RF15 **Envio de Solicitações de Amizade** — MUST HAVE

Envio de pedidos de amizade bilaterais com validação contra duplicações.

RF16 **Gestão de Solicitações (Aceitar/Recusar)** — MUST HAVE

Painel separado para responder pedidos recebidos (aceite via PATCH / recusa) e acompanhar enviados.

RF17 **Desfazimento de Vínculo de Amizade** — MUST HAVE

Opção de remover amizades existentes a qualquer momento por qualquer das partes.

4.5 Módulo de Presença em Tempo Real e Mensageria

RF18 **Controle e Publicação de Presença** — MUST HAVE

Definição de status (`ONLINE` , `BUSY` , `AWAY` , `OFFLINE`) com difusão instantânea via STOMP/Redis.

RF19 **Indicação Automática de Jogo Ativo** — SHOULD HAVE

Atualização automática do perfil ao jogar (ex: "Jogando Cave Story"), visível aos amigos.

RF20 **Chat 1-para-1 em Tempo Real via WebSocket** — SHOULD HAVE

Troca de mensagens de texto privadas entre amigos com persistência relacional e confirmação de leitura.

4.6 Matriz de Resumo e Priorização (MoSCoW)

ID	Nome do Requisito	Módulo	Prioridade
RF01	Autenticação Centralizada (OIDC/PKCE)	Identidade	MUST HAVE
RF02	Cadastro de Nova Conta	Identidade	MUST HAVE
RF03	Controle de Acesso RBAC	Identidade	MUST HAVE
RF04	Encerramento Seguro (Logout)	Identidade	MUST HAVE
RF05	Visualização e Edição de Perfil	Gestão de Contas	MUST HAVE
RF06	Alteração de E-mail	Gestão de Contas	SHOULD HAVE
RF07	Exclusão Permanente de Conta	Gestão de Contas	SHOULD HAVE
RF08	Consulta ao Catálogo de Jogos	Jogos Sob Demanda	MUST HAVE
RF09	Provisionamento Dinâmico K8s	Jogos Sob Demanda	MUST HAVE
RF10	Acompanhamento de Prontidão (Probes)	Jogos Sob Demanda	MUST HAVE
RF11	Game Stage (Execução Embutida)	Jogos Sob Demanda	MUST HAVE
RF12	Heartbeat e Idle Cleanup	Jogos Sob Demanda	MUST HAVE
RF13	Encerramento Manual de Instância	Jogos Sob Demanda	MUST HAVE
RF14	Diretório de Jogadores Paginado	Rede Social	MUST HAVE
RF15	Envio de Solicitações de Amizade	Rede Social	MUST HAVE
RF16	Gestão de Solicitações Recebidas/Enviadas	Rede Social	MUST HAVE
RF17	Desfazimento de Amizade	Rede Social	MUST HAVE
RF18	Controle e Publicação de Presença	Tempo Real	MUST HAVE
RF19	Indicação Automática de Jogo Ativo	Tempo Real	SHOULD HAVE
RF20	Chat Privado via WebSocket	Tempo Real	SHOULD HAVE

5. Requisitos Não-Funcionais

5.1 Desempenho e Eficiência

RNF01 **Tempo de Inicialização de Jogo** MUST HAVE

O tempo entre o clique do usuário e o carregamento do jogo no navegador não deve ultrapassar **10 segundos** com imagens em cache no cluster local.

RNF02 **Tempo de Resposta das APIs REST** MUST HAVE

O tempo de resposta no percentil 95 (p95) para consultas de catálogo, amizades e diretório deve ser inferior a **200ms**.

RNF03 **Latência na Propagação de Presença** SHOULD HAVE

Eventos de presença e status de jogo via WebSocket/Redis devem ser entregues aos clientes conectados em menos de **500ms**.

5.2 Segurança e Privacidade

RNF04 **Comunicação Cifrada Ponta a Ponta (TLS)** MUST HAVE

100% do tráfego (HTTP, WebSocket e instâncias de jogos) deve utilizar HTTPS/WSS com certificados emitidos automaticamente pelo `Cert-Manager`.

RNF05 **Isolamento e Proteção OIDC por Instância** MUST HAVE

Cada instância de jogo deve conter um proxy OIDC (OAuth2-Proxy), garantindo que apenas o usuário solicitante acesse a URL da sessão.

RNF06 **Proteção contra Ameaças Web (OWASP Top 10)** MUST HAVE

Mitigação de vulnerabilidades XSS, CSRF, injeção de comandos e proteção de cookies (`HttpOnly`, `SameSite=Lax`, `Secure`).

5.3 Confiabilidade, Disponibilidade e Tolerância a Falhas

RNF07 **Autocura de Contêineres (Self-Healing)** MUST HAVE

O Kubernetes deve monitorar os pods via *Liveness* e *Readiness Probes*, reiniciando contêineres que entrem em falha.

RNF08 **Isolamento e Statelessness das Partidas** MUST HAVE

Instâncias de jogo devem ser descartáveis; a queda de uma sessão individual não pode afetar outras partidas ou a plataforma central.

5.4 Usabilidade e Acessibilidade

RNF09 **Design Responsivo Multidispositivo** MUST HAVE

A interface deve se adaptar com fidelidade desde smartphones (360px) até monitores desktop e ultra-wide (4K).

RNF10 **Feedback Visual e Loading States** MUST HAVE

Todas as operações assíncronas devem apresentar esqueletos (skeletons), indicadores de progresso ou spinners animados.

5.5 Portabilidade e Manutenibilidade da Infraestrutura

RNF11 **Infraestrutura como Código (IaC) e Reprodutibilidade** MUST HAVE

Toda a infraestrutura (Kind, Helm, Traefik, Keycloak, Postgres, Redis) deve ser reprodutível via `Makefiles` do repositório.

RNF12 **Documentação Interativa de APIs (Swagger UI)** SHOULD HAVE

Endpoints REST devem ser documentados e testáveis pelo Swagger UI do SpringDoc OpenAPI em `/swagger-ui.html`.

RNF13 **Conformidade Arquitetural (ArchUnit)** MUST HAVE

O backend deve passar nas validações automáticas de camadas arquiteturais do ArchUnit durante a fase de testes (`gradle check`).

5.6 Matriz de Resumo dos Requisitos Não-Funcionais

ID	Requisito Não-Funcional	Categoria ISO 25010	Critério de Aceitação	Prioridade
RNF01	Tempo de Inicialização de Jogo	Desempenho	≤ 10s no cluster	MUST HAVE
RNF02	Tempo de Resposta REST	Desempenho	p95 ≤ 200ms	MUST HAVE
RNF03	Latência de Presença (STOMP)	Desempenho	≤ 500ms	SHOULD HAVE
RNF04	Comunicação Cifrada (TLS)	Segurança	100% HTTPS/WSS	MUST HAVE
RNF05	Isolamento OIDC por Partida	Segurança	OAuth2-Proxy por Pod	MUST HAVE
RNF06	Mitigações OWASP	Segurança	Cookies e Sanitização	MUST HAVE
RNF07	Autocura e Probes K8s	Confiabilidade	Liveness/Readiness	MUST HAVE
RNF08	Statelessness das Partidas	Confiabilidade	Isolamento total	MUST HAVE
RNF09	Responsividade Multidispositivo	Usabilidade	360px a 4K	MUST HAVE
RNF10	Feedback e Loading States	Usabilidade	Skeletons/Spinners	MUST HAVE
RNF11	Automação de Infraestrutura	Manutenibilidade	Makefile + Helm 100%	MUST HAVE
RNF12	Documentação Swagger UI	Manutenibilidade	Swagger UI ativo	SHOULD HAVE
RNF13	Regras ArchUnit	Manutenibilidade	Passar no Gradle check	MUST HAVE